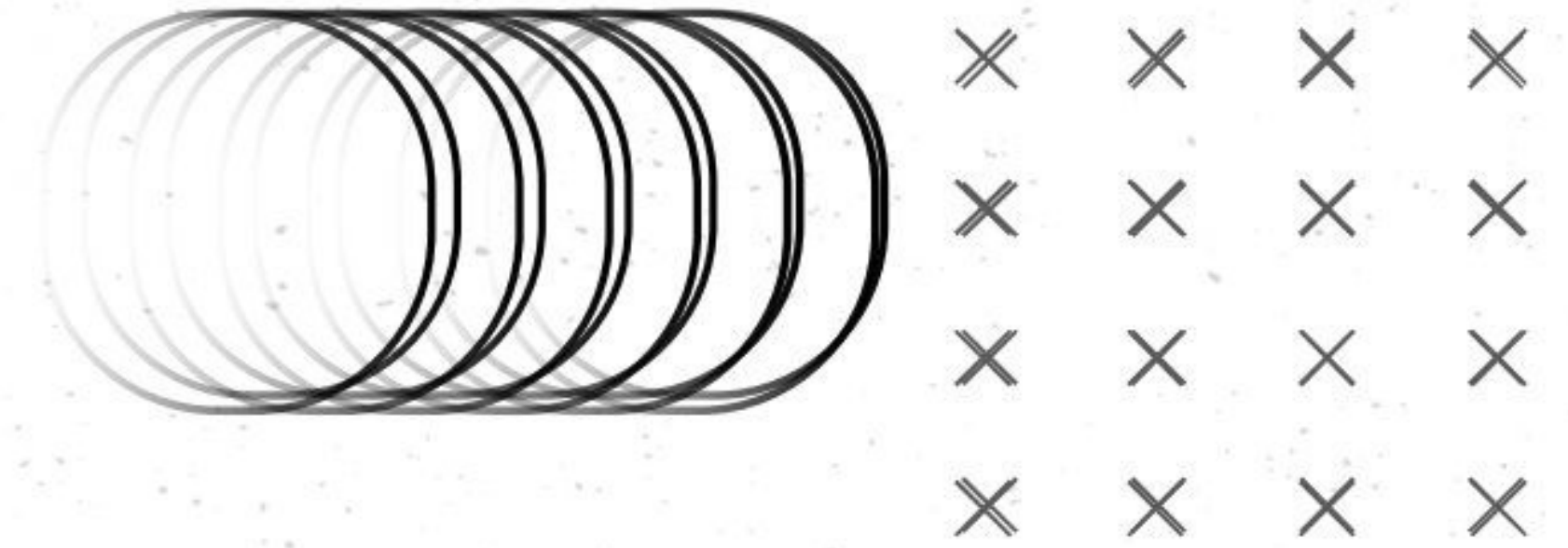




Data structure

النخعي

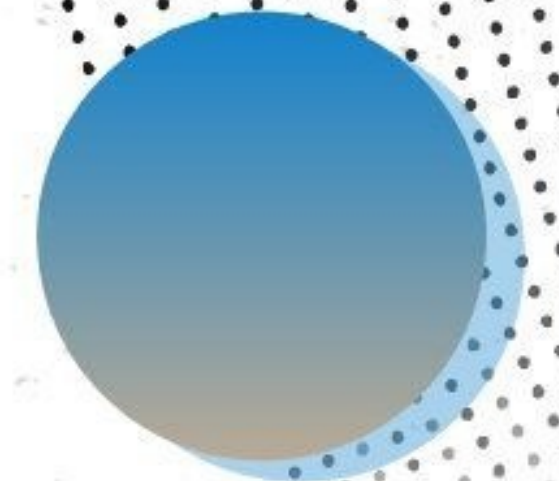
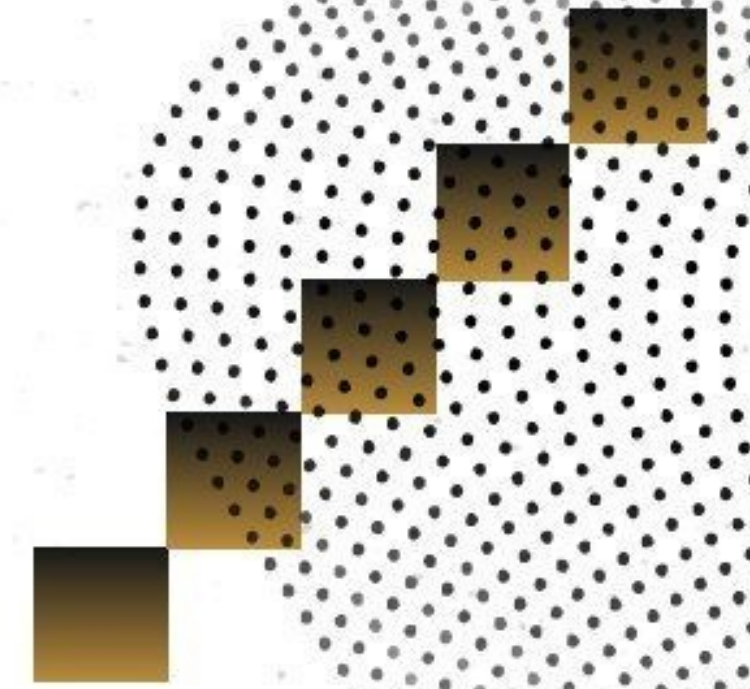
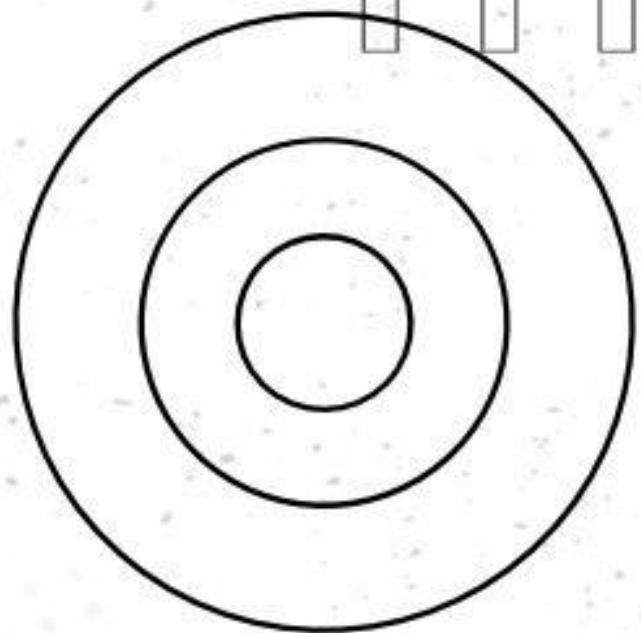
LECTURE " 4 " : Recursion



Quick Recall

- Algorithms solve problems step-by-step
- Functions help organize logic
- Stack follows LIFO behavior

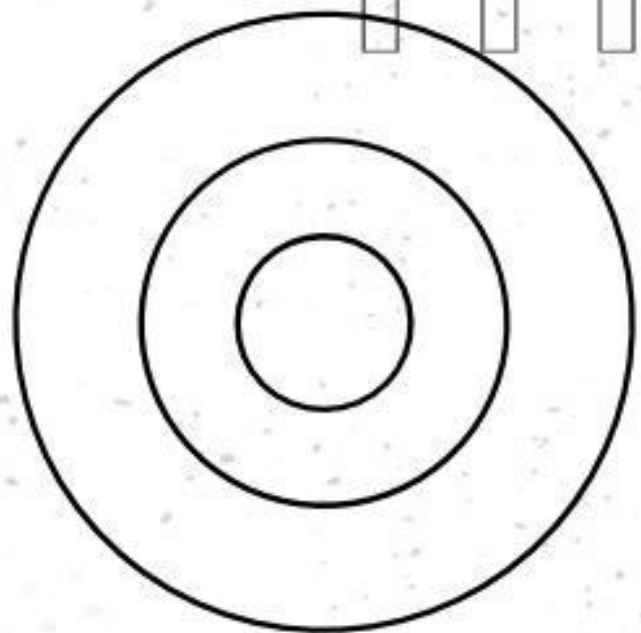
< T 0 >



What happens
when a function is
called

Memory is allocated
Function waits until
completion

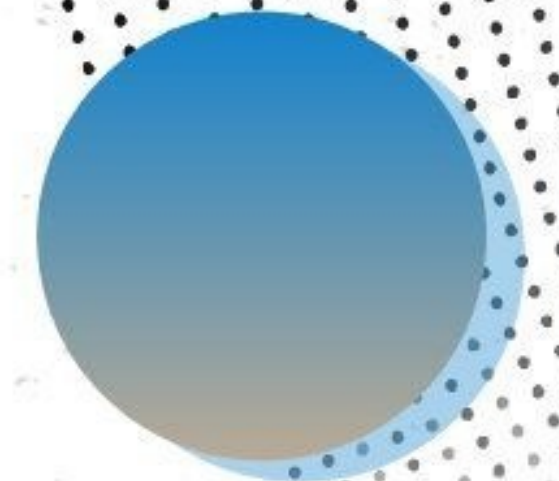
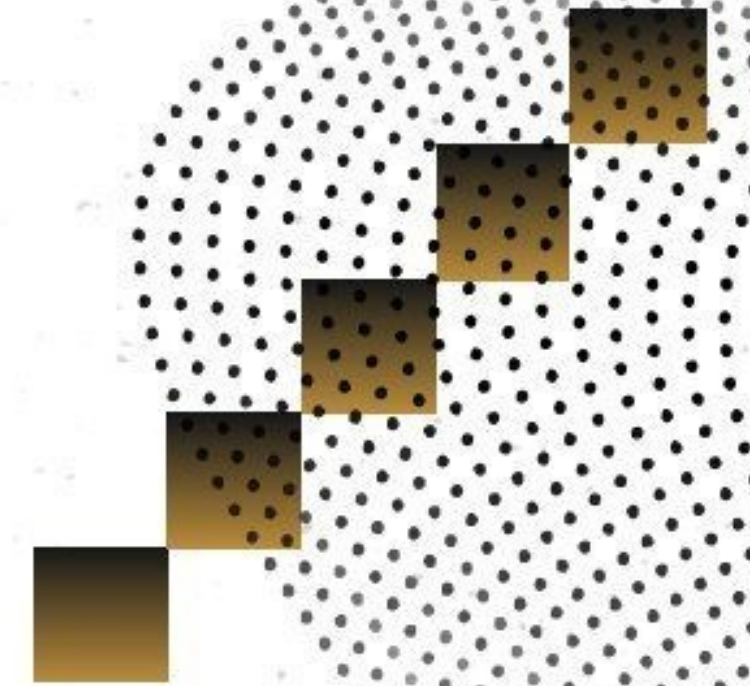
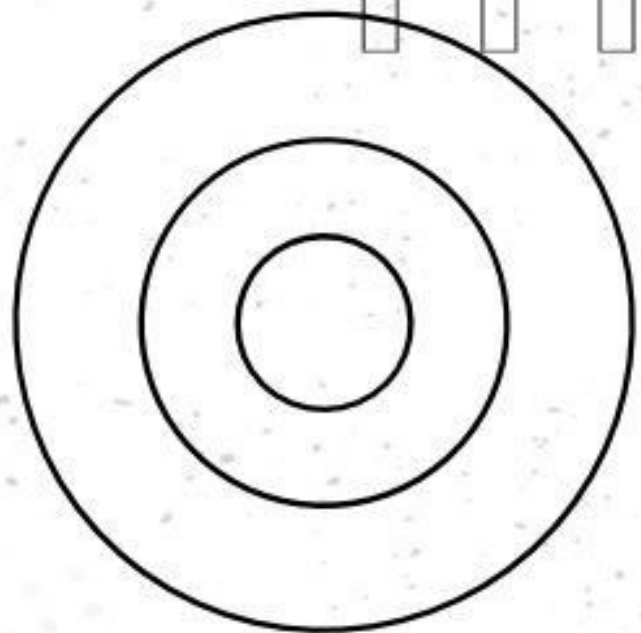
< T 0 >



What is Recursion

Recursion is a
programming technique
where:

- A function calls itself
- The problem is divided into smaller versions of the same problem



Real-Life Analogies

Mirror facing mirror
Russian dolls

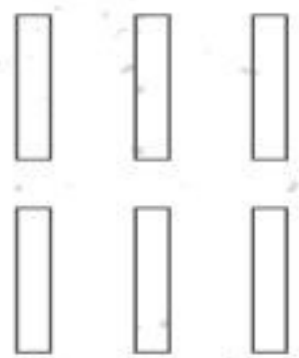
 Folder inside folder

 Tree branches splitting into
smaller branches

< T 0 >

Why do we use recursion

- Simplifies complex problems
 - Makes some algorithms easier to write
 - Natural solution for hierarchical structures



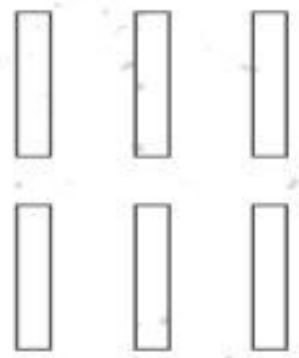


Base Case

Base Case = Stopp Condition

A recursive function must
know:

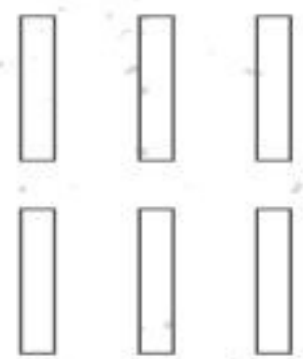
- When to stop
- When the problem be
simple enough



Recursive Case

The function:

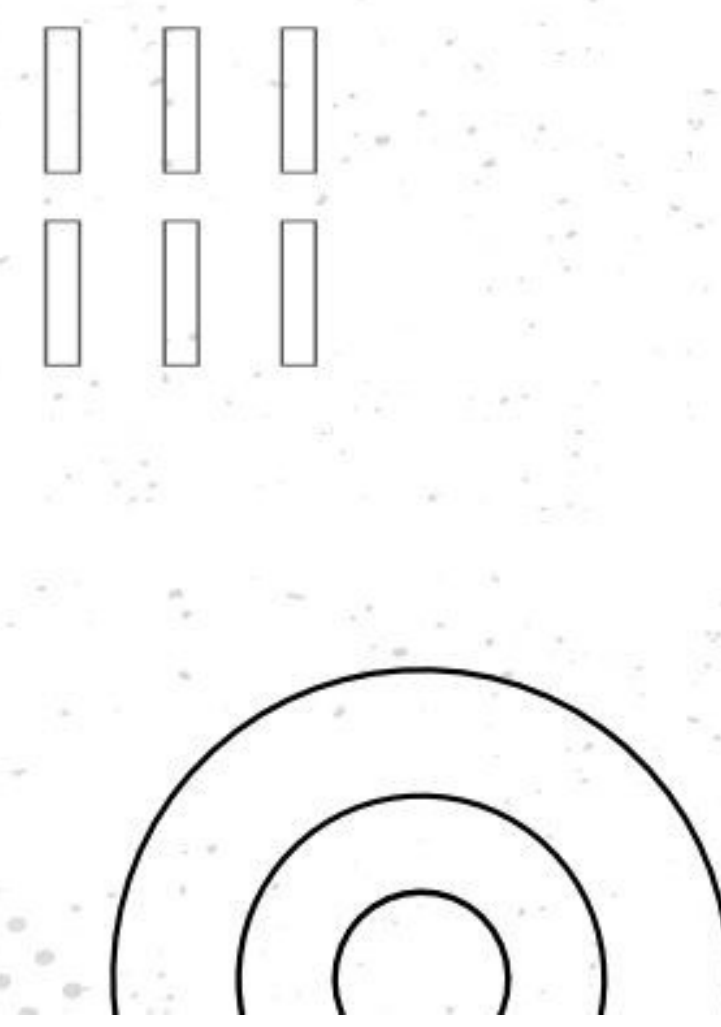
- Calls itself
- Works on a smaller problem
- Moves closer to the base case





Recursive Case

“Recursion works by reducing the problem step-by-step.”



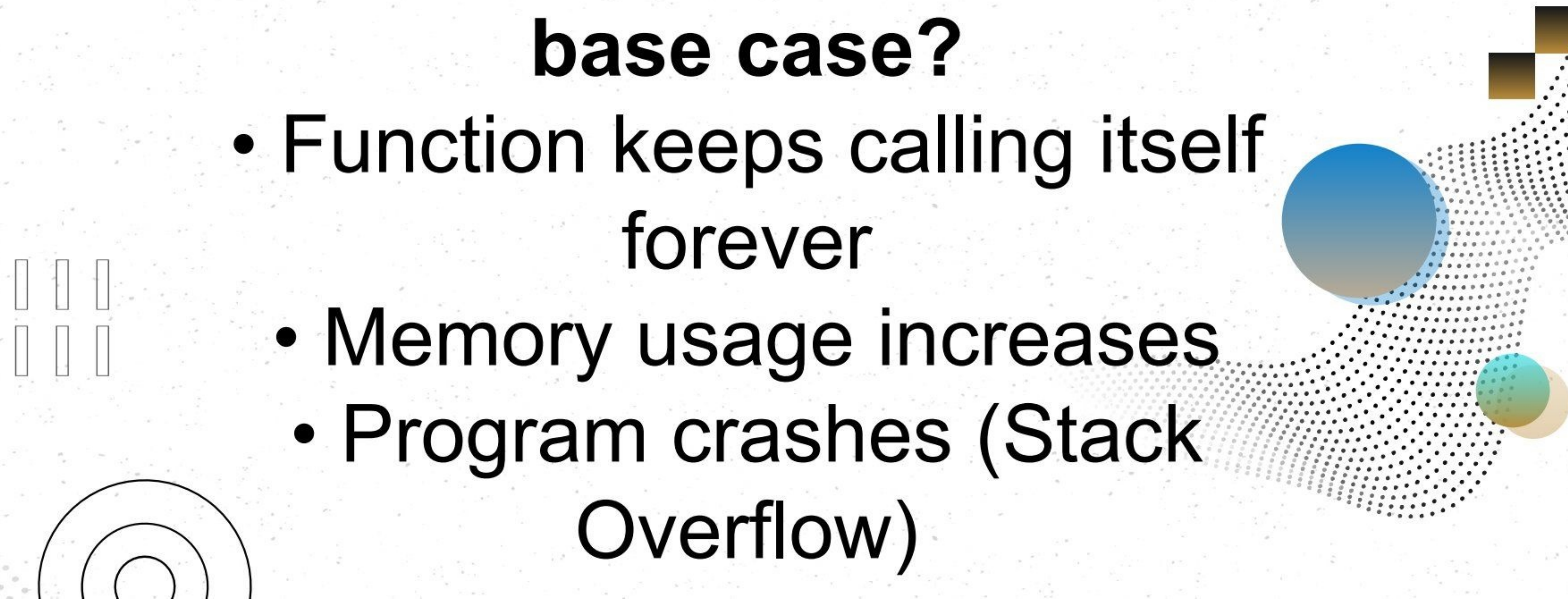
Example:
factorial(5)
→ factorial(4)
→ factorial(3)

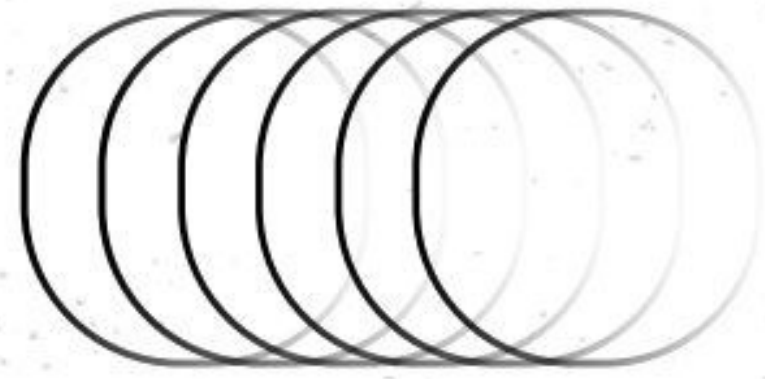




Infinite Recursion Problem

**What happens without a
base case?**

- Function keeps calling itself forever
 - Memory usage increases
 - Program crashes (Stack Overflow)
- 



Recursion Flow Visualization

- Function calls itself repeatedly
- Calls are stored in stack order
- Execution returns backward after base case ●



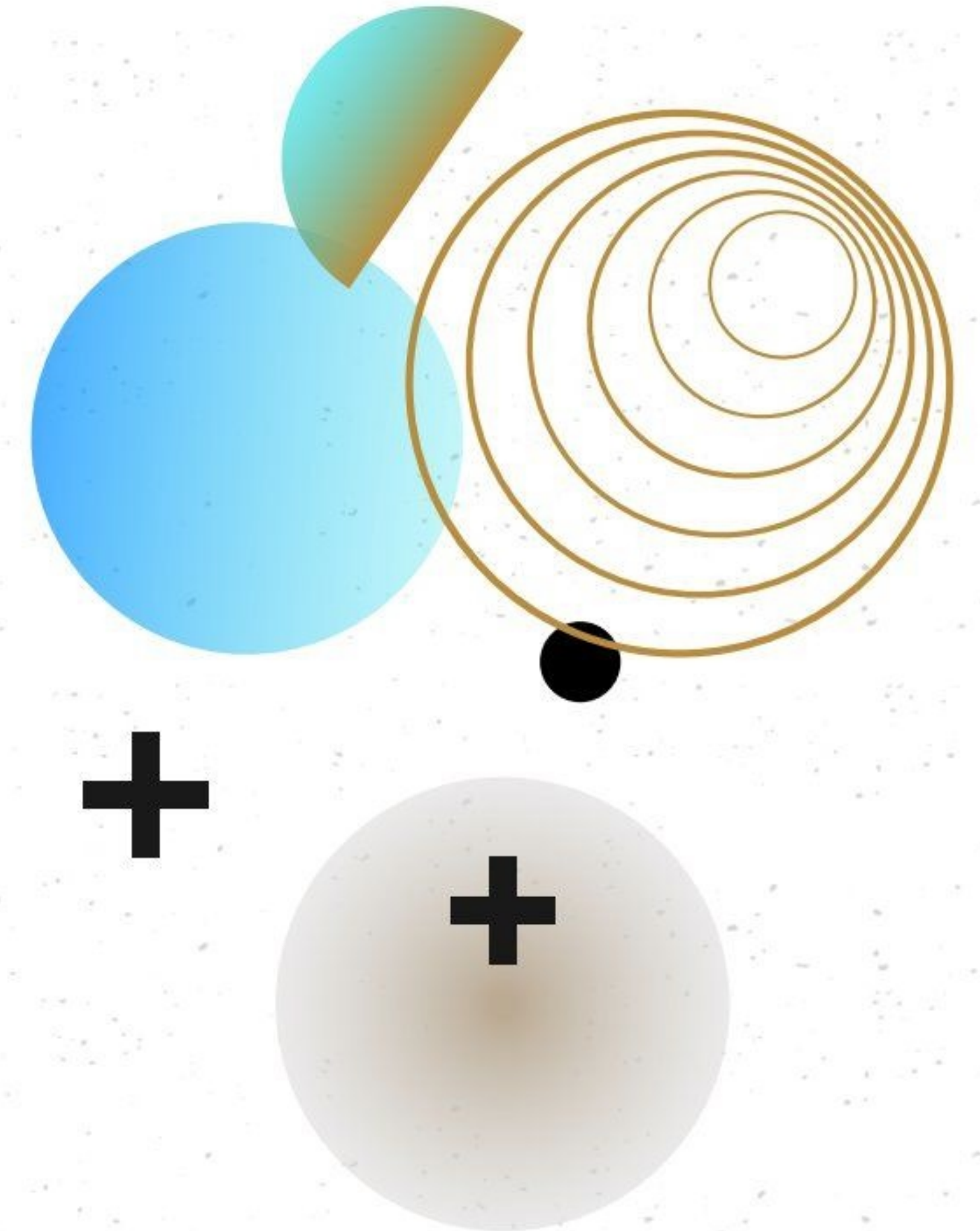
+

+

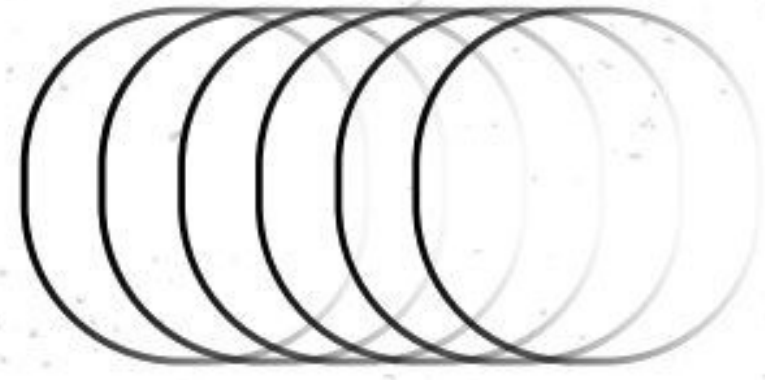


< 0 1 >

Recursion Flow Visualization

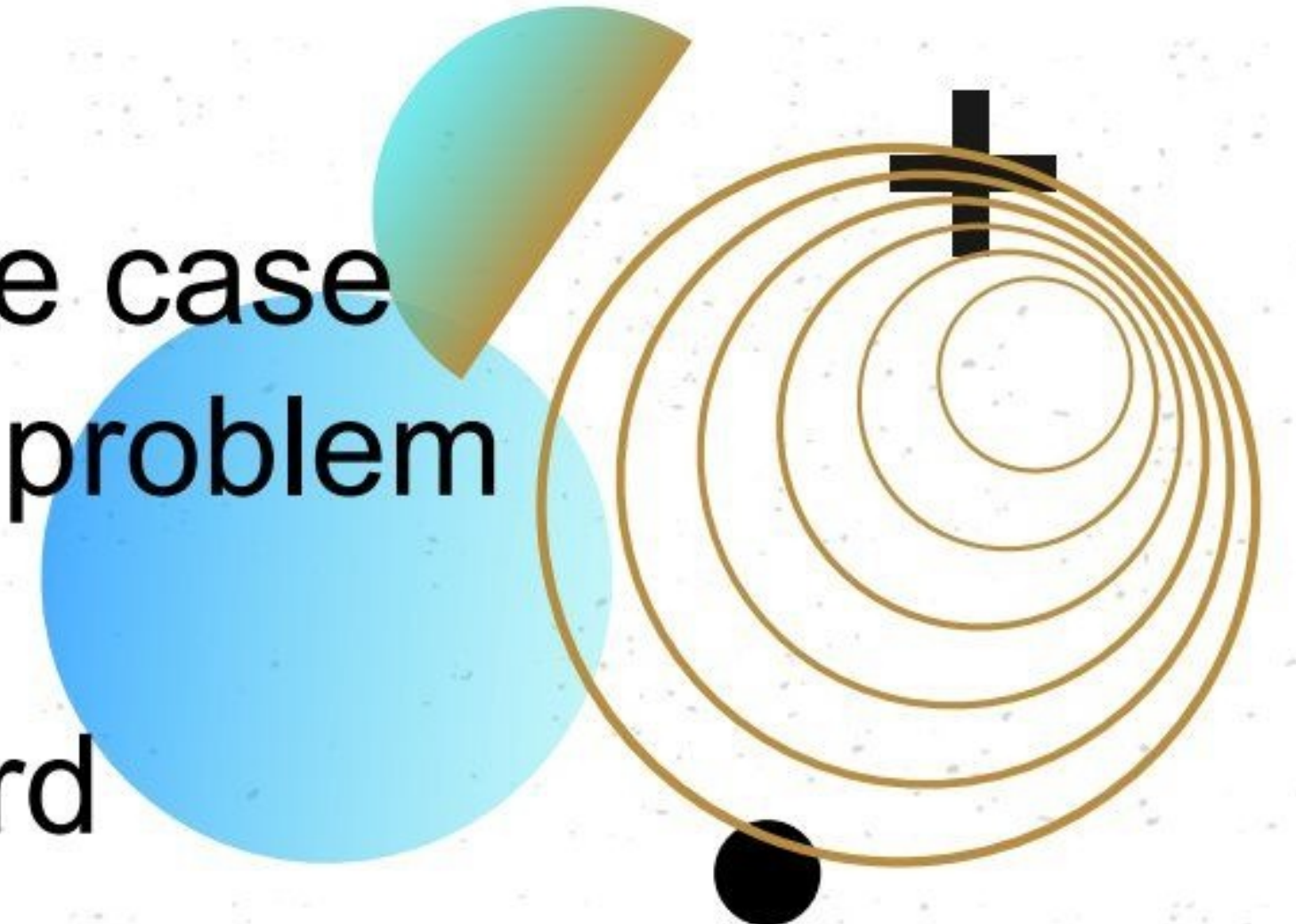


< 0 1 >



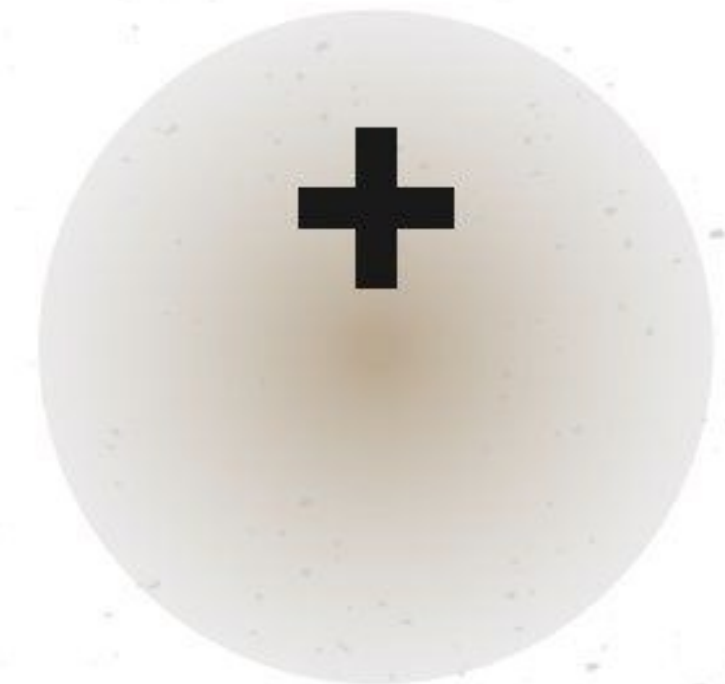
Key Rules of Recursion

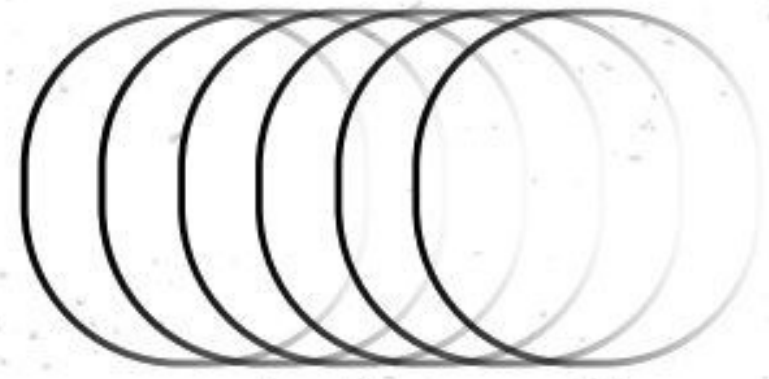
- ✓ Must have a base case
- ✓ Must reduce the problem size
- ✓ Must move toward stopping condition



Otherwise:

- Infinite recursion
- Stack overflow
- Program failure





Factorial Concept

The factorial of a number n is:

$$n! = n \times (n-1) \times (n-2) \times \dots \times 1$$
$$n! = n \times (n-1) \times (n-2) \times \dots \times 1$$

Examples:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$
$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

$$2! = 2 \times 1 = 2$$

$$3! = 3 \times 2 \times 1 = 6$$
$$3! = 3 \times 2 \times 1 = 6$$

$$6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 720$$

×

×

×

×

×

×

×

×

×

×

×

×

×

×

×

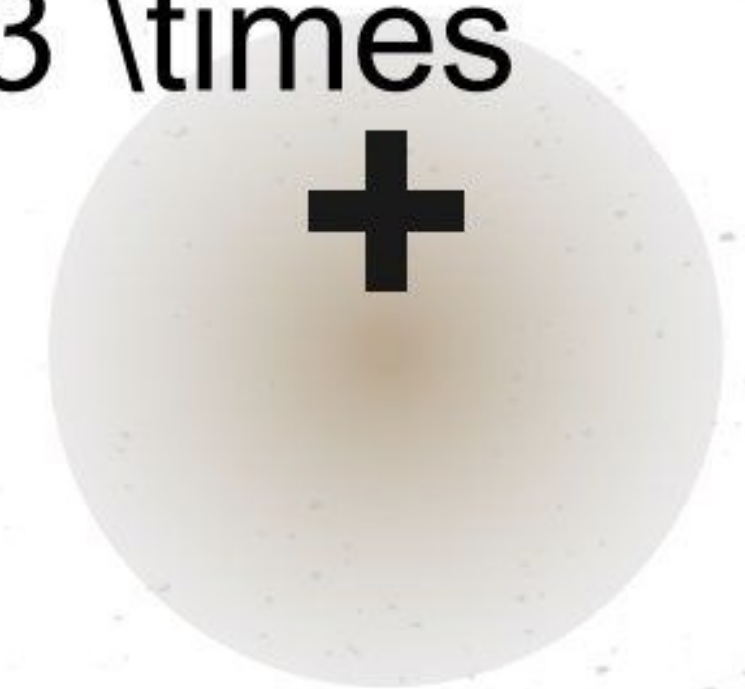
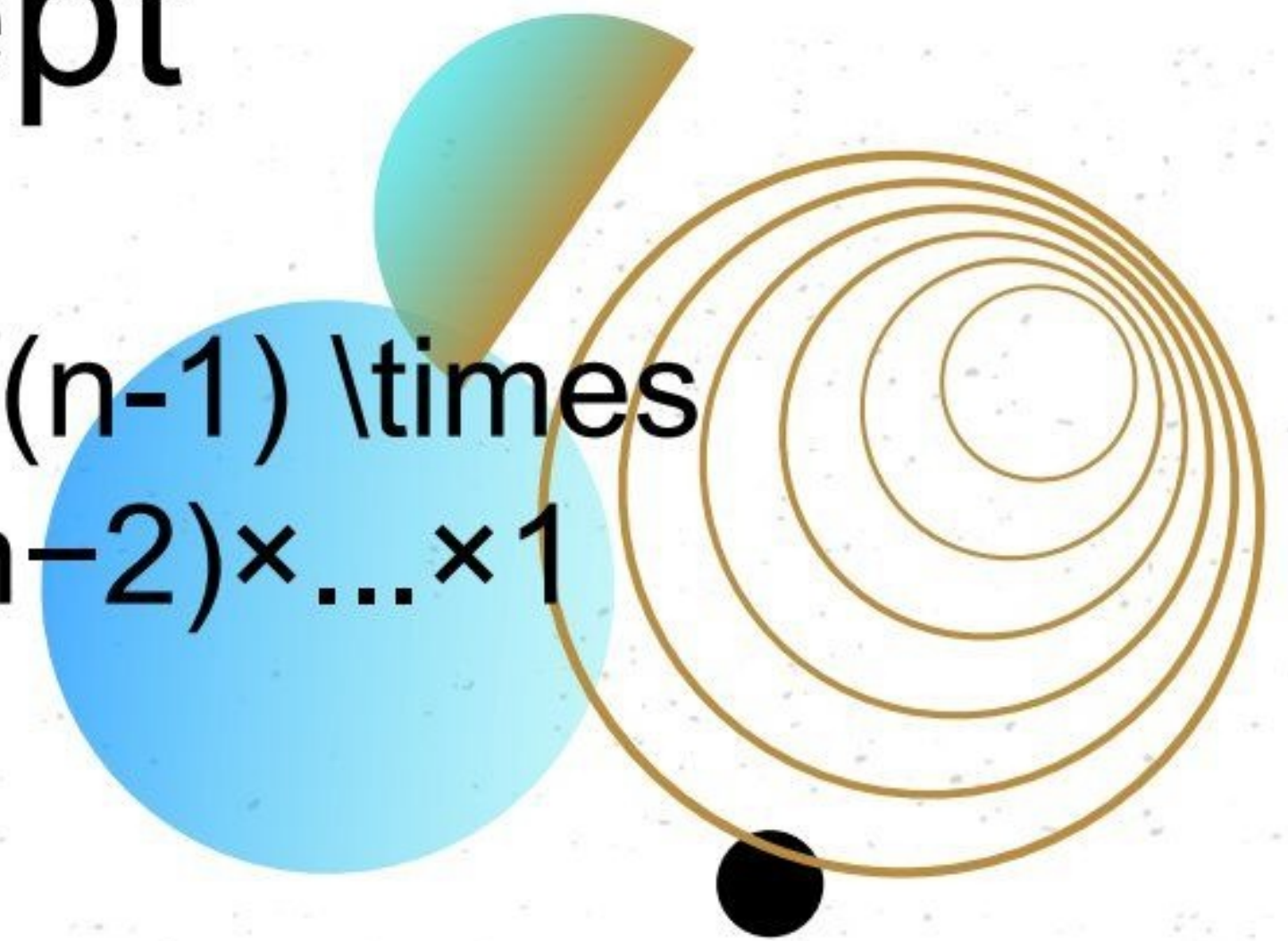
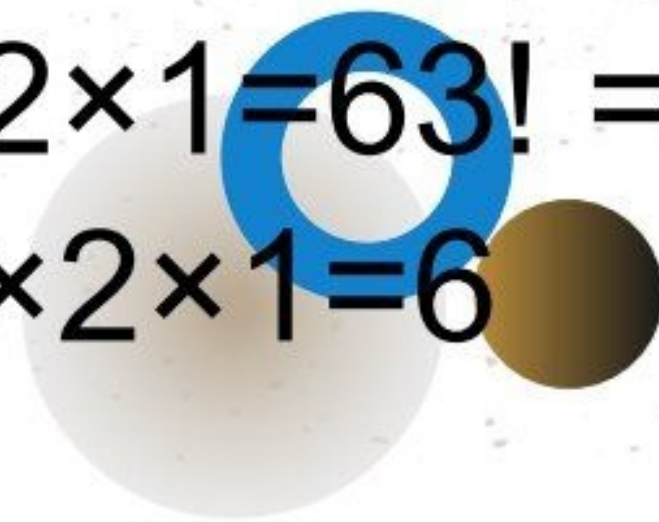
×

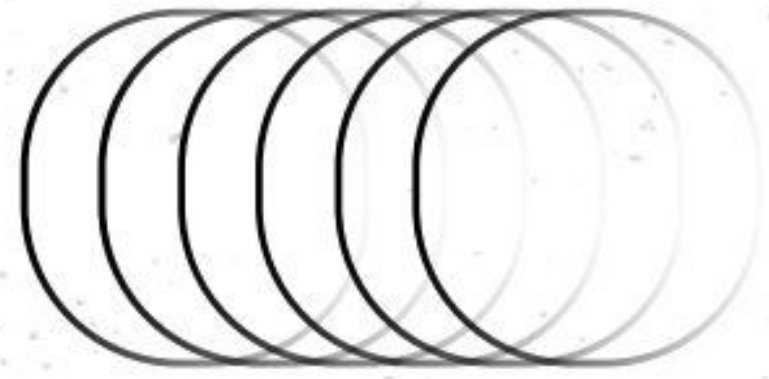
×

×

×

×





Recursive Thinking

Instead of writing the full multiplication:

$$5! = 5 \times 4! \quad 5! = 5 \times 4!$$

$$4! = 4 \times 3! \quad 4! = 4 \times 3!$$

$$3! = 3 \times 2! \quad 3! = 3 \times 2!$$



+

+

Same problem repeated with smaller input



< 0 1 >

Factorial Pseudocode

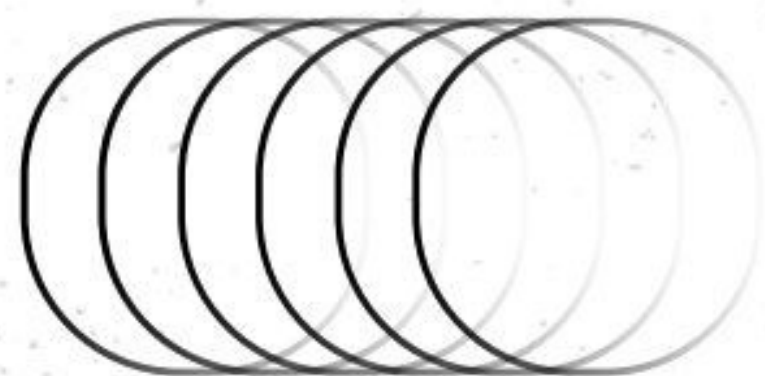
FUNCTION
factorial(n):

IF $n == 0$:
RETURN 1

RETURN $n *$

Factorial C# Code:
int Factorial(int n)

```
{  
    if (n == 0)  
        return 1;  
  
    return n * Factorial(n -  
1);  
}
```



Step-by-Step Recursive Calls

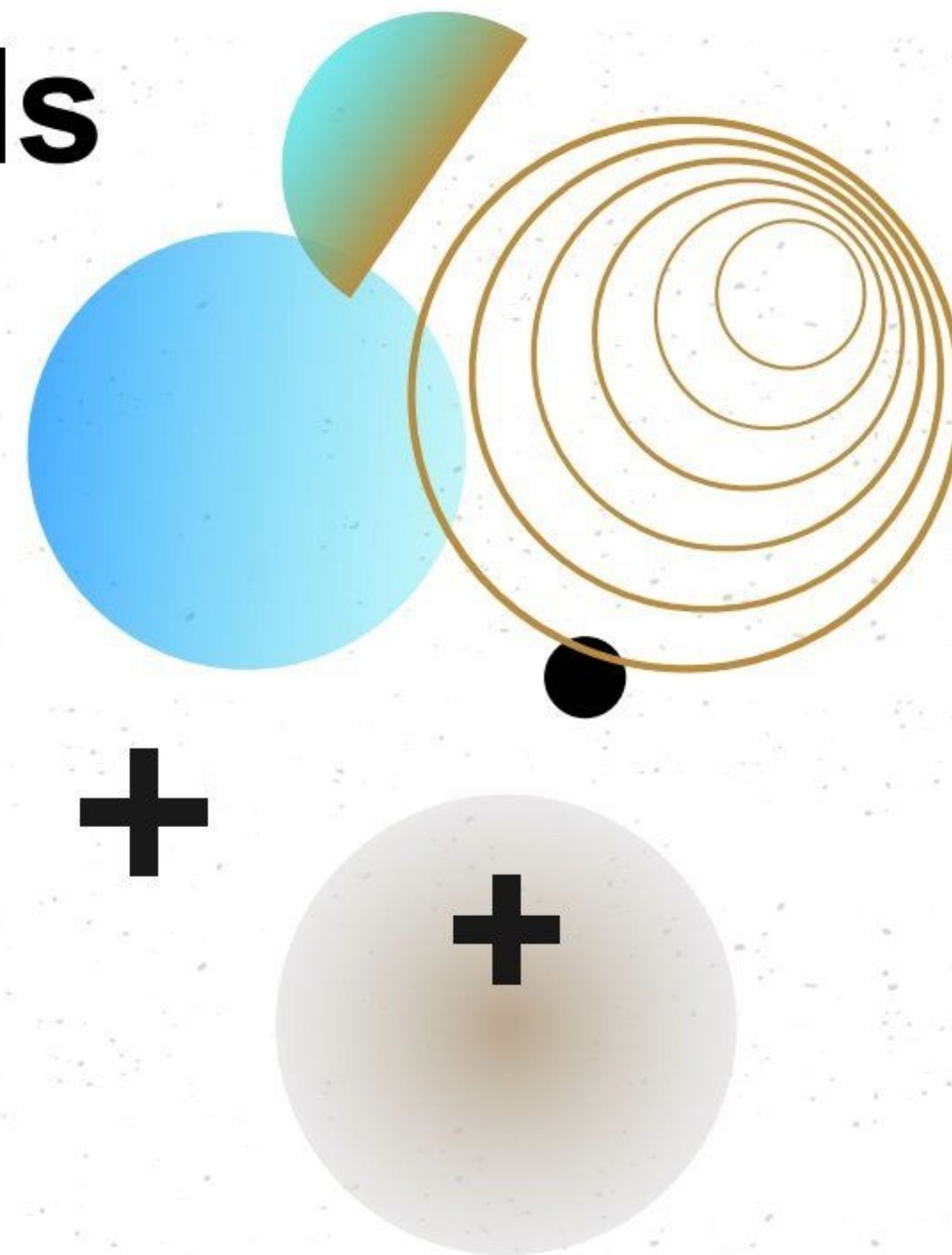
$$\text{Factorial}(5) \\ = 5 \times \text{Factorial}(4)$$

$$= 5 \times (4 \times \text{Factorial}(3))$$

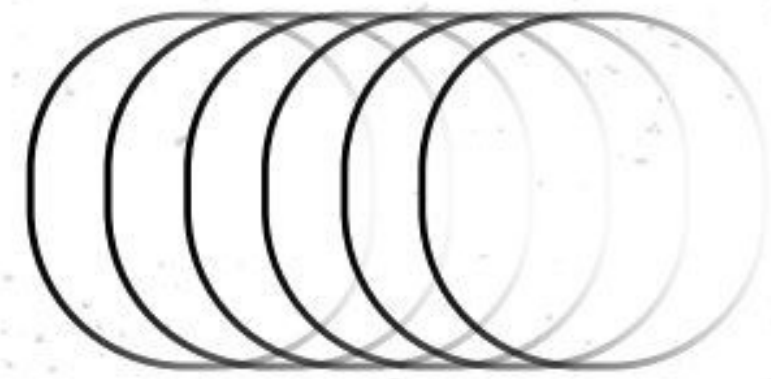
$$= 5 \times (4 \times (3 \times \text{Factorial}(2)))$$

$$= 5 \times (4 \times (3 \times (2 \times \text{Factorial}(1))))$$

< 0 1 >



× × × ×
× × × ×
× × × ×
× × × ×



Base Case Reached

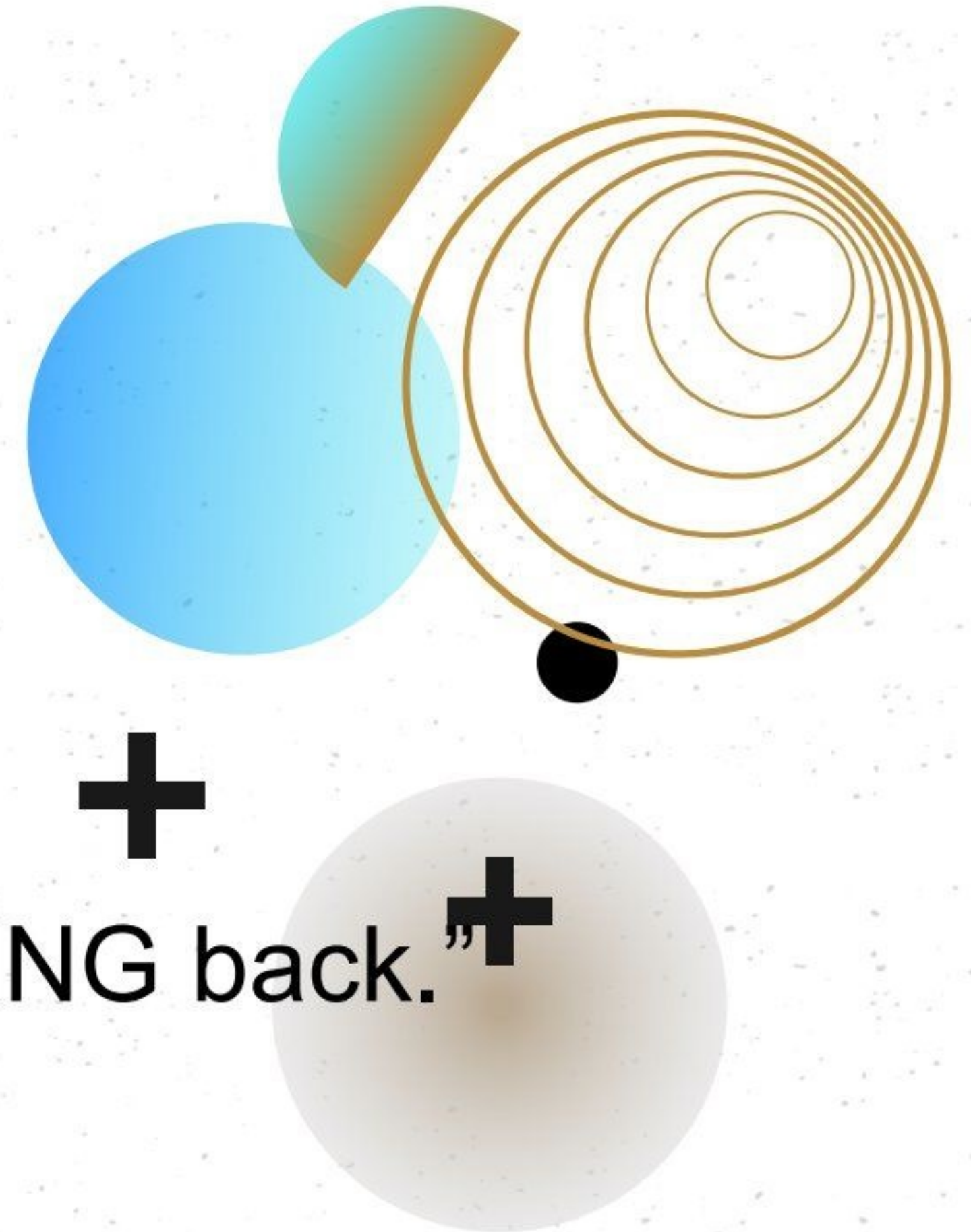
Factorial(1)
= 1

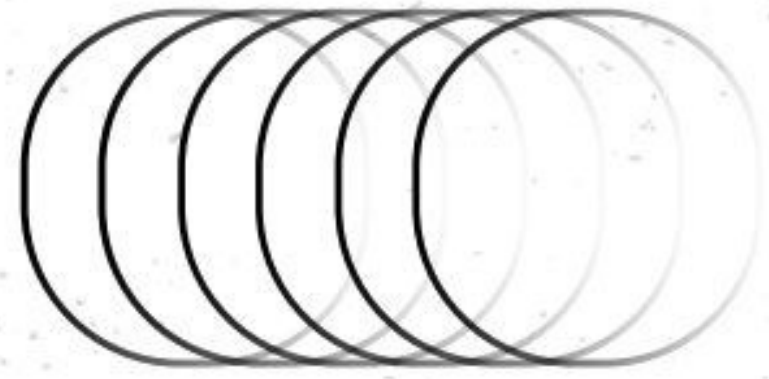
Now recursion stops

“Now the function starts RETURNING back.”



< 0 1 >





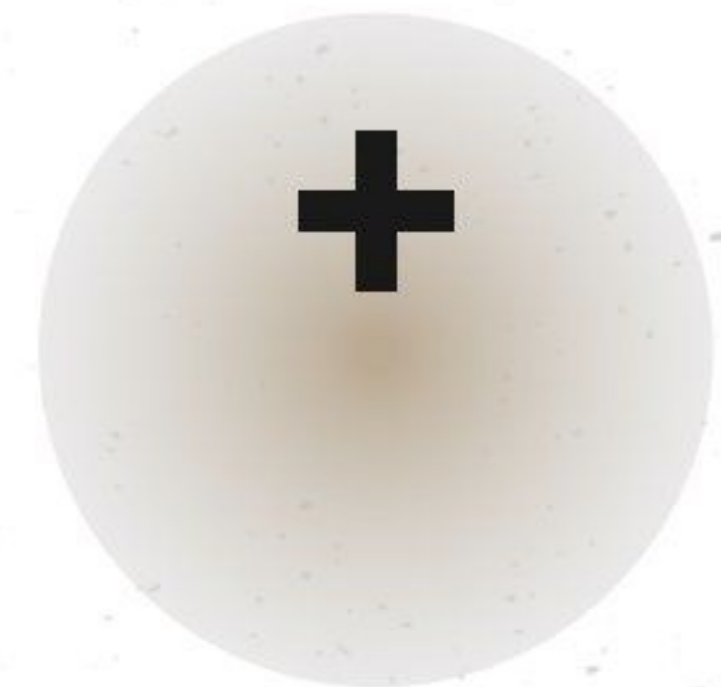
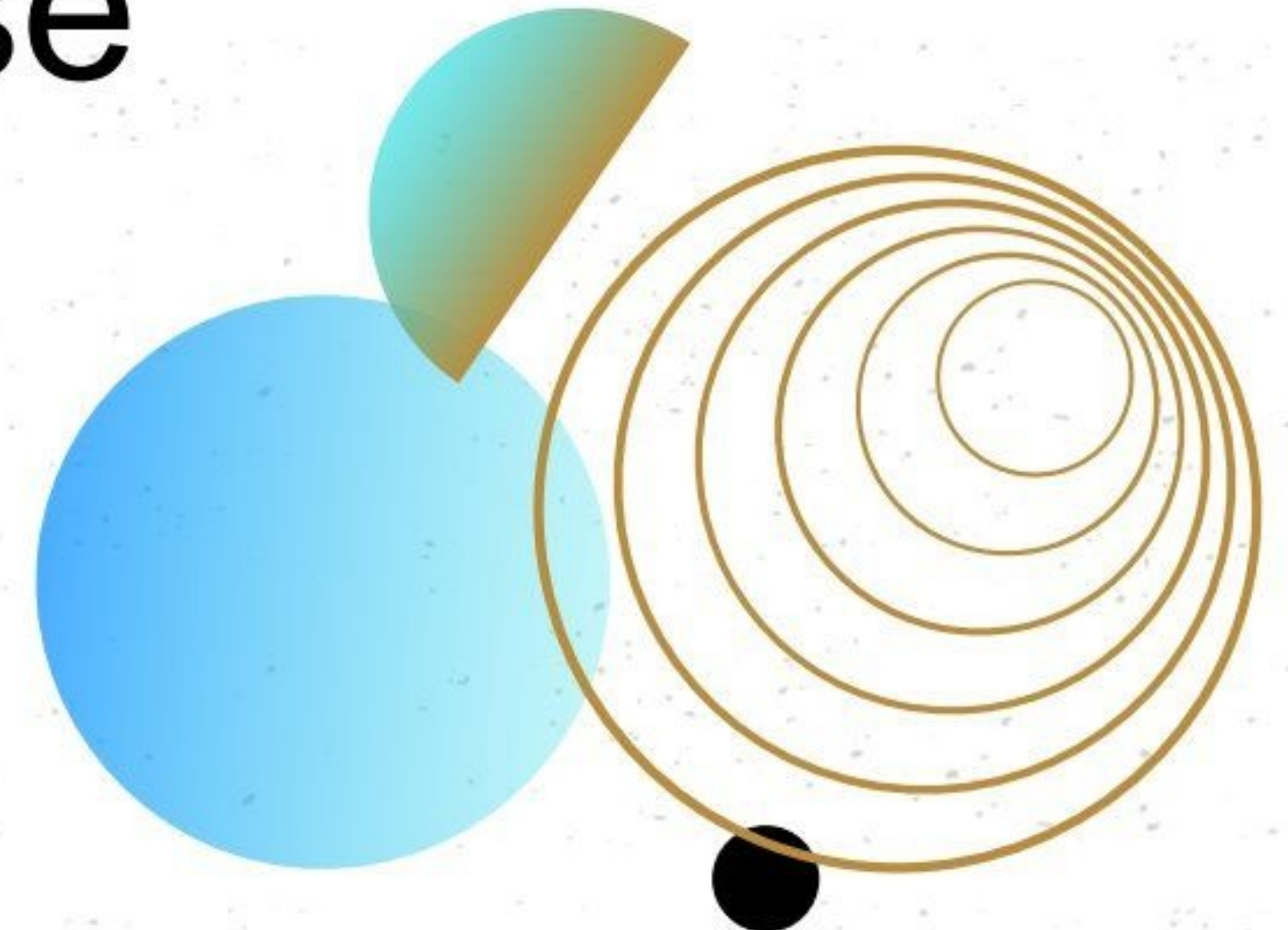
Returning Phase

$$\text{Factorial}(2) \\ = 2 \times 1 = 2$$

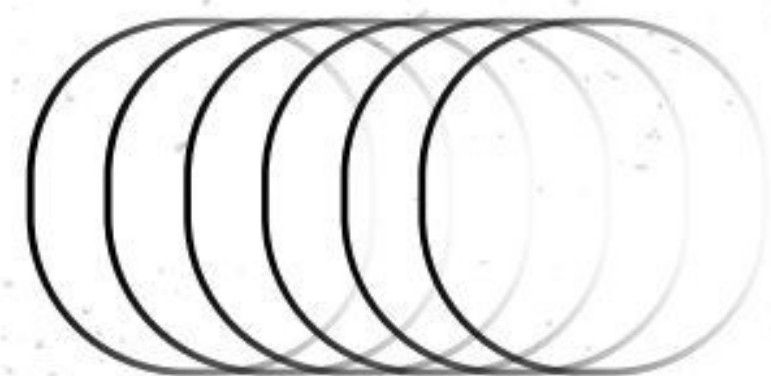
$$\text{Factorial}(3) \\ = 3 \times 2 = 6$$

$$\text{Factorial}(4) \\ = 4 \times 6 = 24$$

$$\text{Factorial}(5) \\ = 5 \times 24 = 120+$$



< 0 1 >



Recursion Stack Visualization

Factorial(5):

Factorial(5)

Factorial(4)

Factorial(3)

Factorial(2)

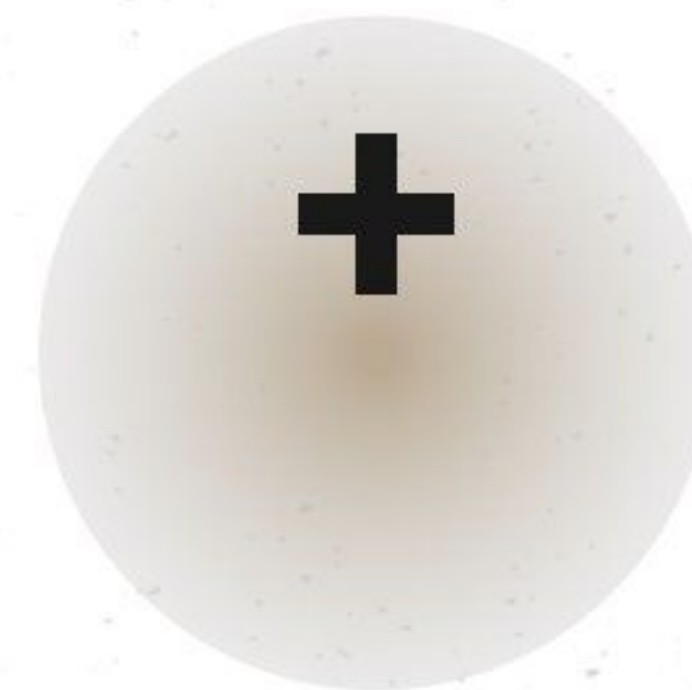
Factorial(1)

× × × ×
× × × ×
× × × ×
× × × ×



+

+

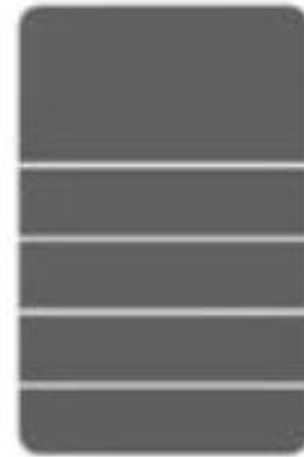


< 0 1 >

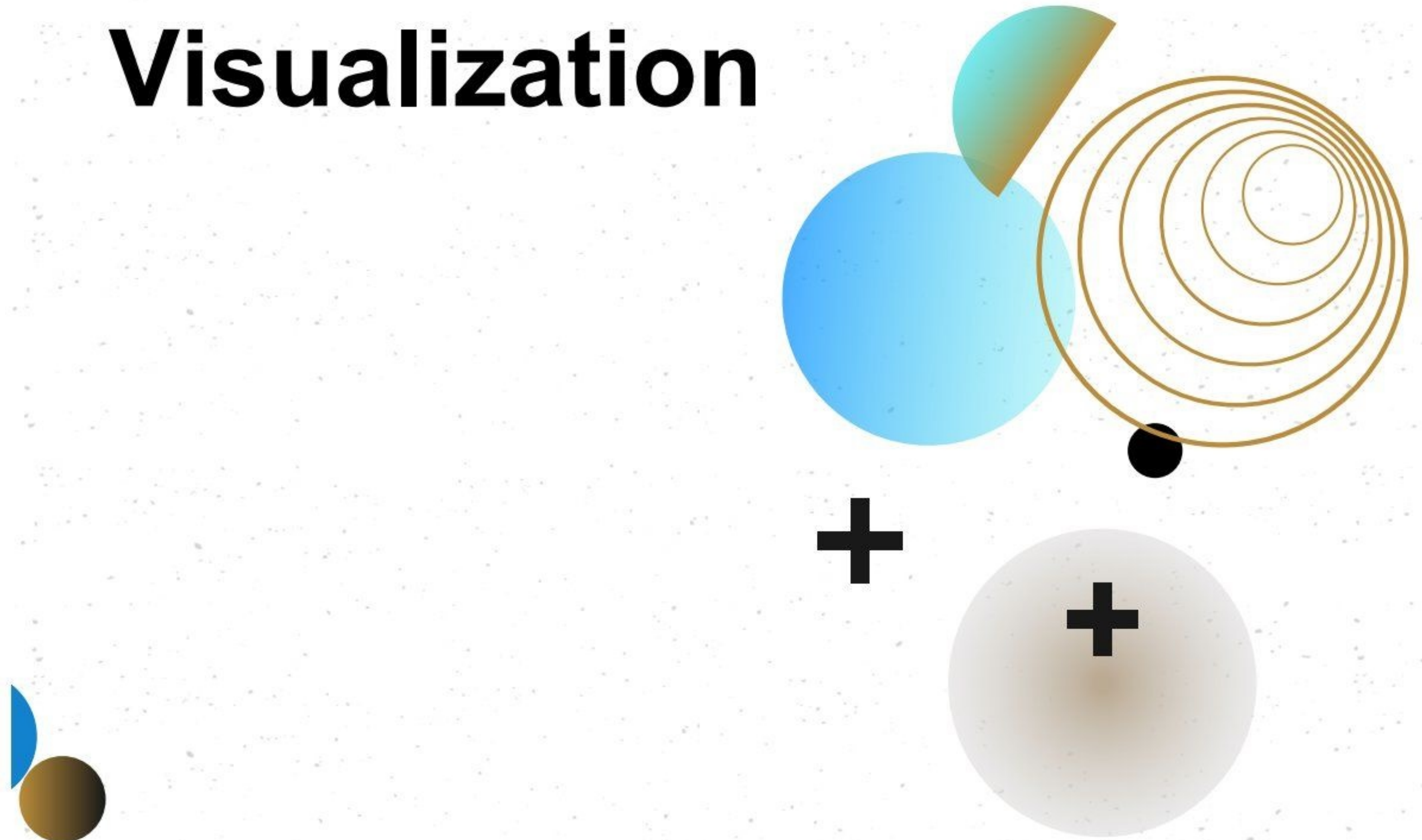
How “**Recursion**” Works?

```
function factorial(n) {  
  if (n === 0 || n === 1) {  
    return 1;  
  } else {  
    return n * factorial(n - 1);  
  }  
}
```

Call Stack



Recursion Stack Visualization



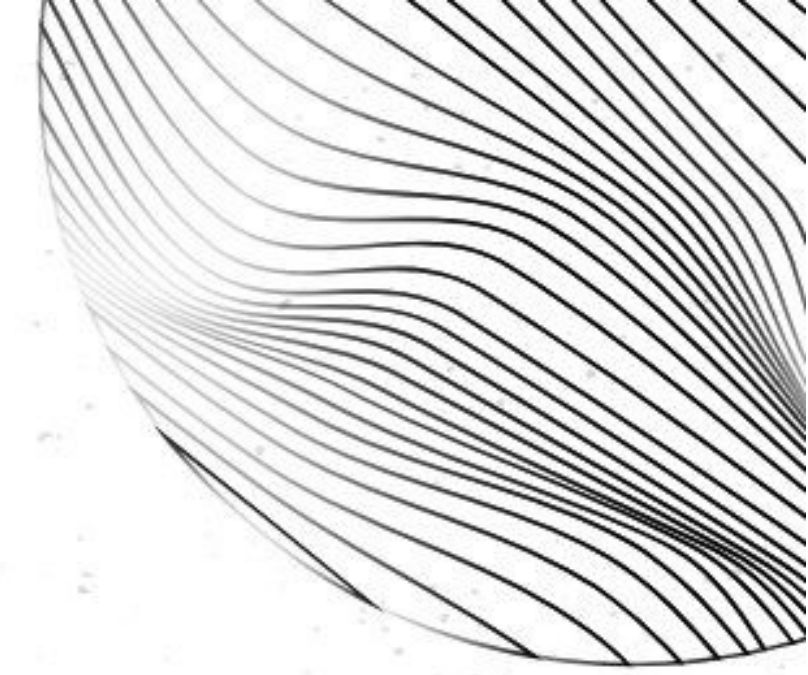
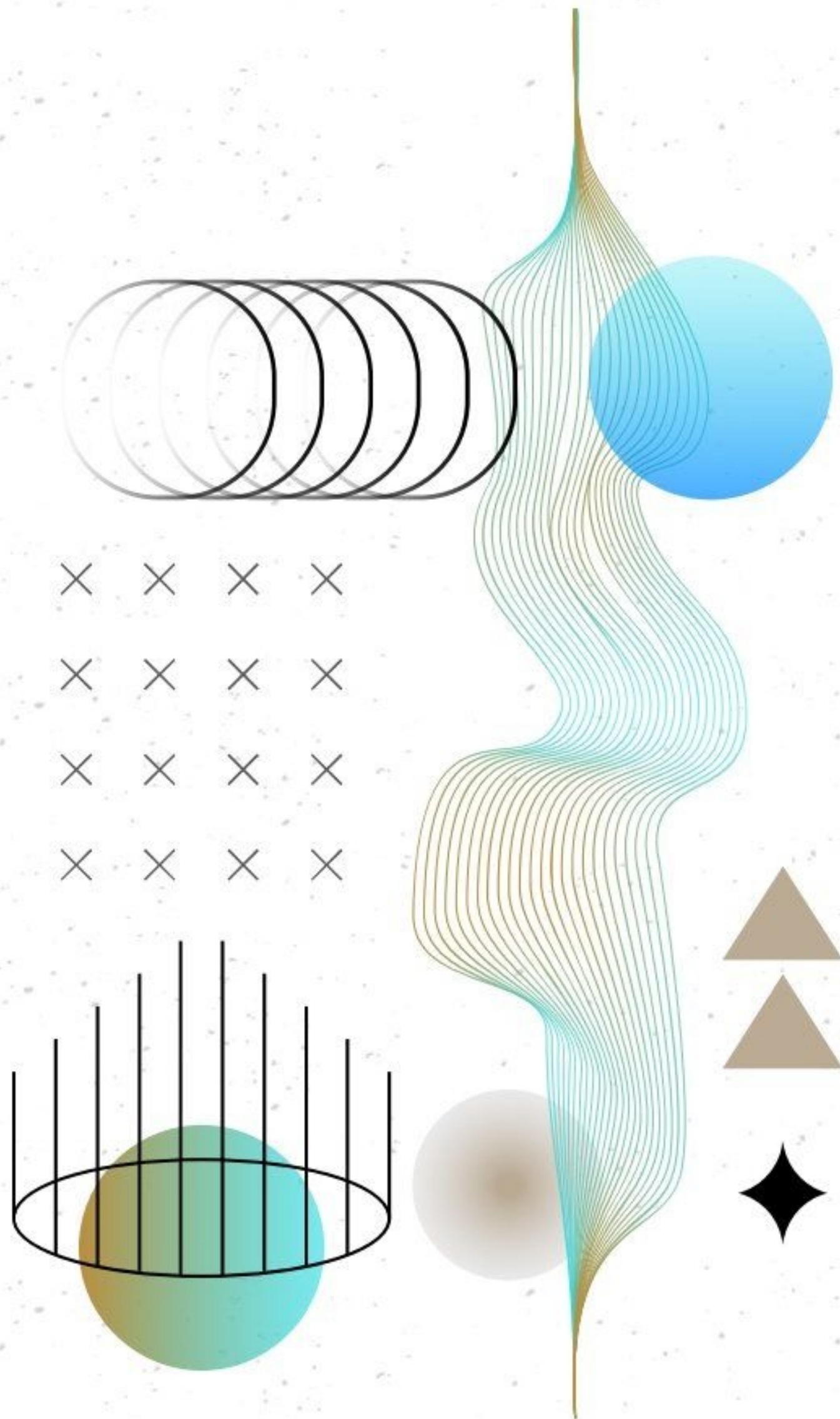
< 0 1 >

Fibonacci Recursion Example

Fibonacci sequence:
0, 1, 1, 2, 3, 5, 8, 13, ..

Each number equals:
 $F(n) = F(n-1) + F(n-2)$

“Each value depends on the two previous values Unlike factorial... Fibonacci creates TWO recursive calls.



Fibonacci

Pseudocode

FUNCTION Fibonacci(n):

IF $n == 0$:
 RETURN 0

IF $n == 1$:
 RETURN 1

RETURN Fibonacci($n-1$) + Fibonacci($n-2$)

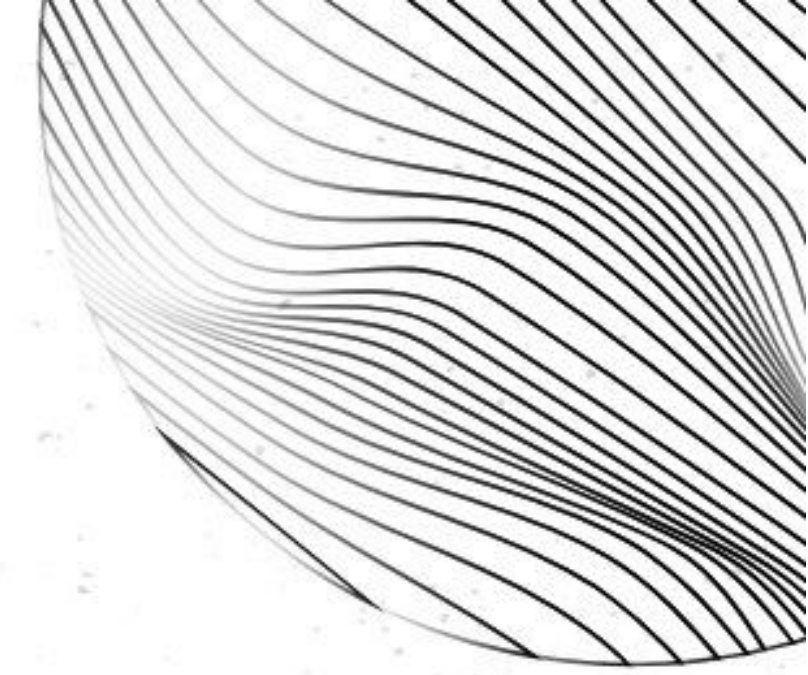
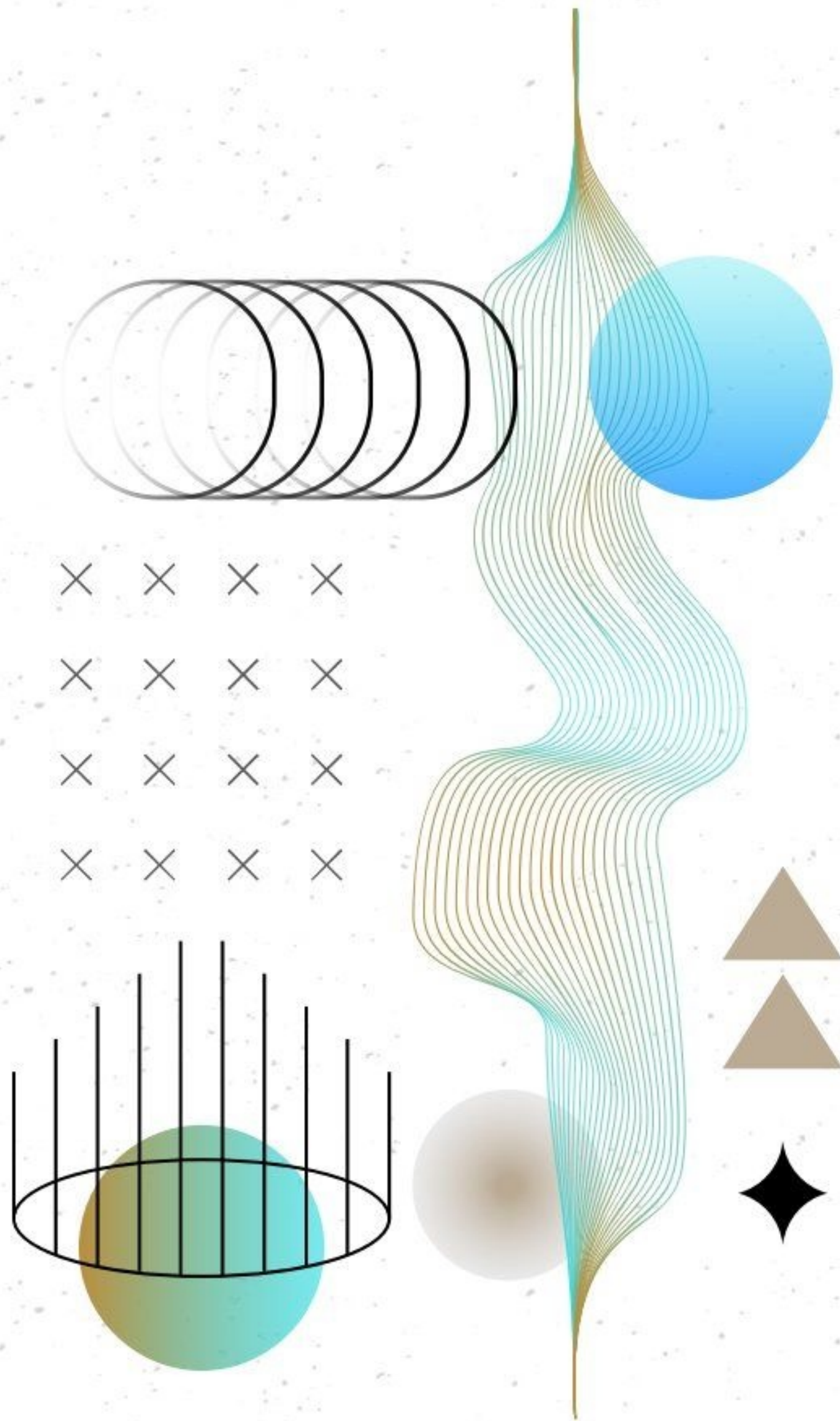
Fibonacci c# code

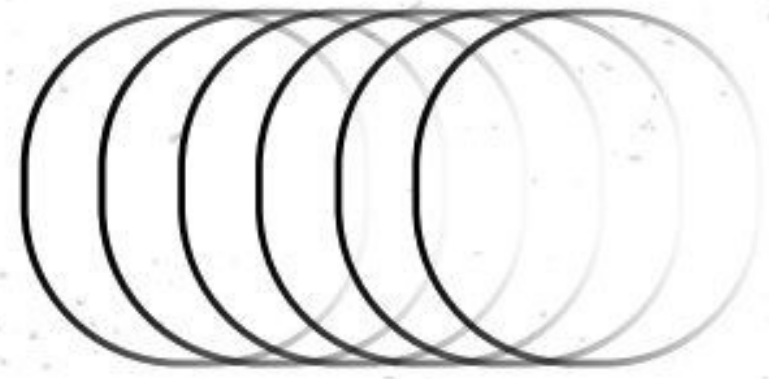
```
int Fibonacci(int n)
{
```

```
    if (n == 0)
        return 0;
```

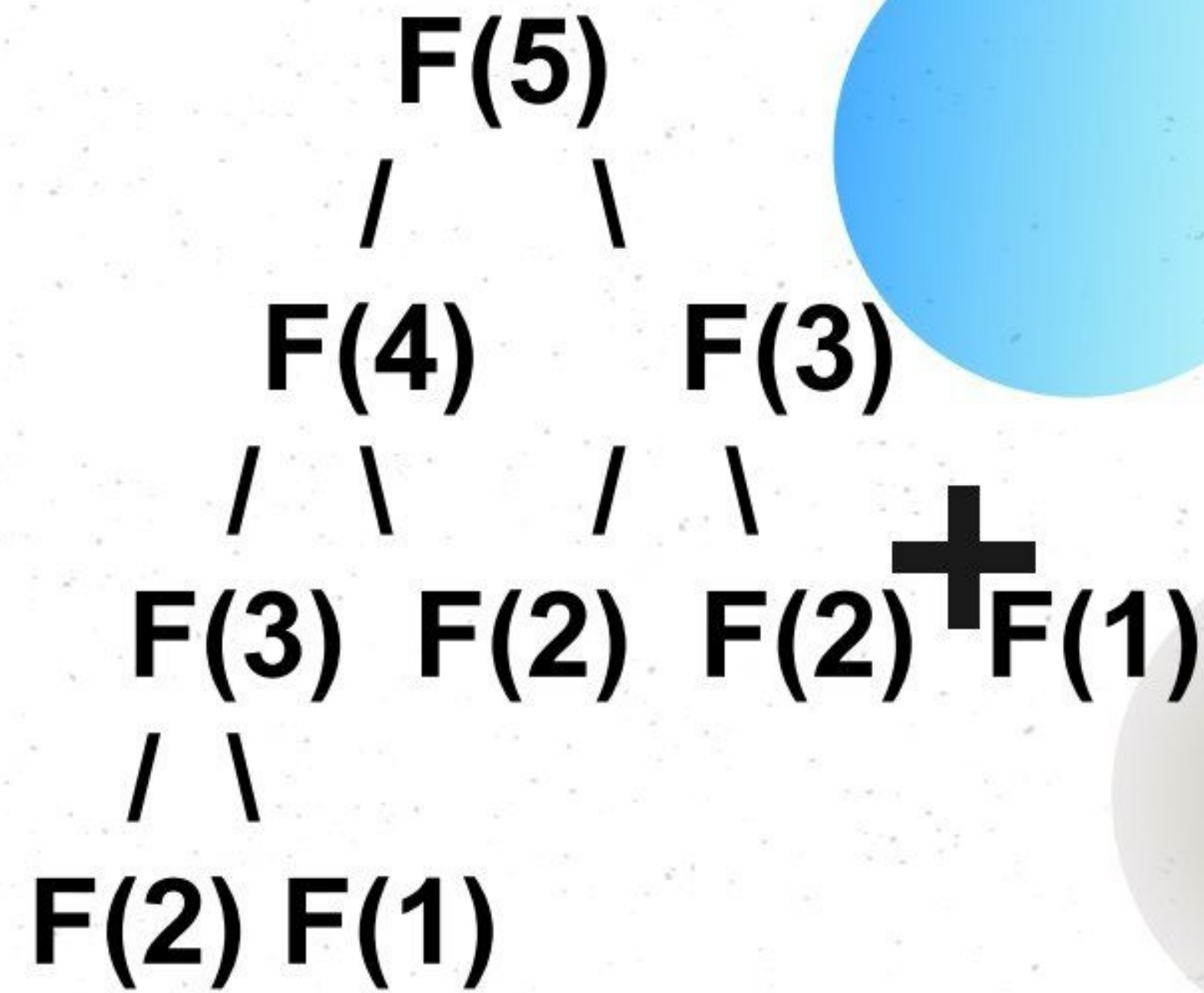
```
    if (n == 1)
        return 1;
```

```
    return Fibonacci(n - 1) + Fibonacci(n - 2);
}
```

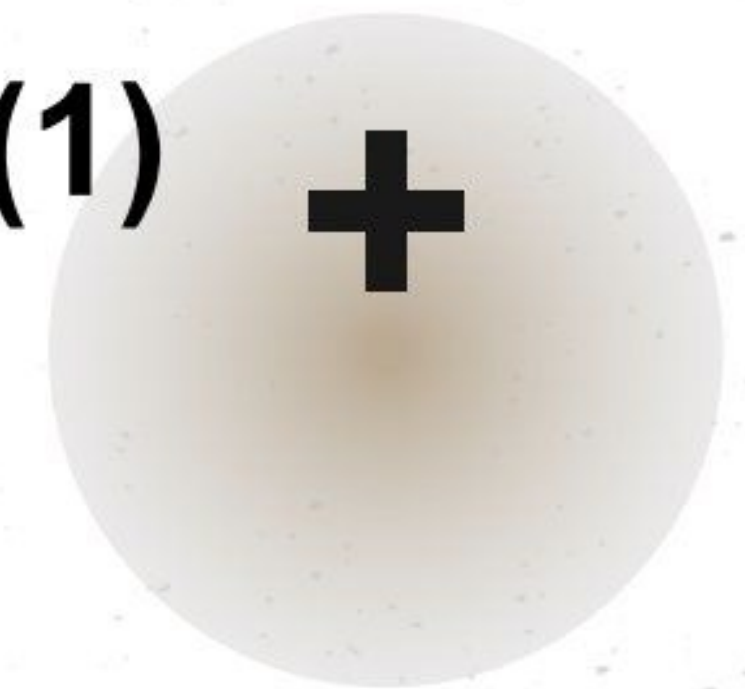
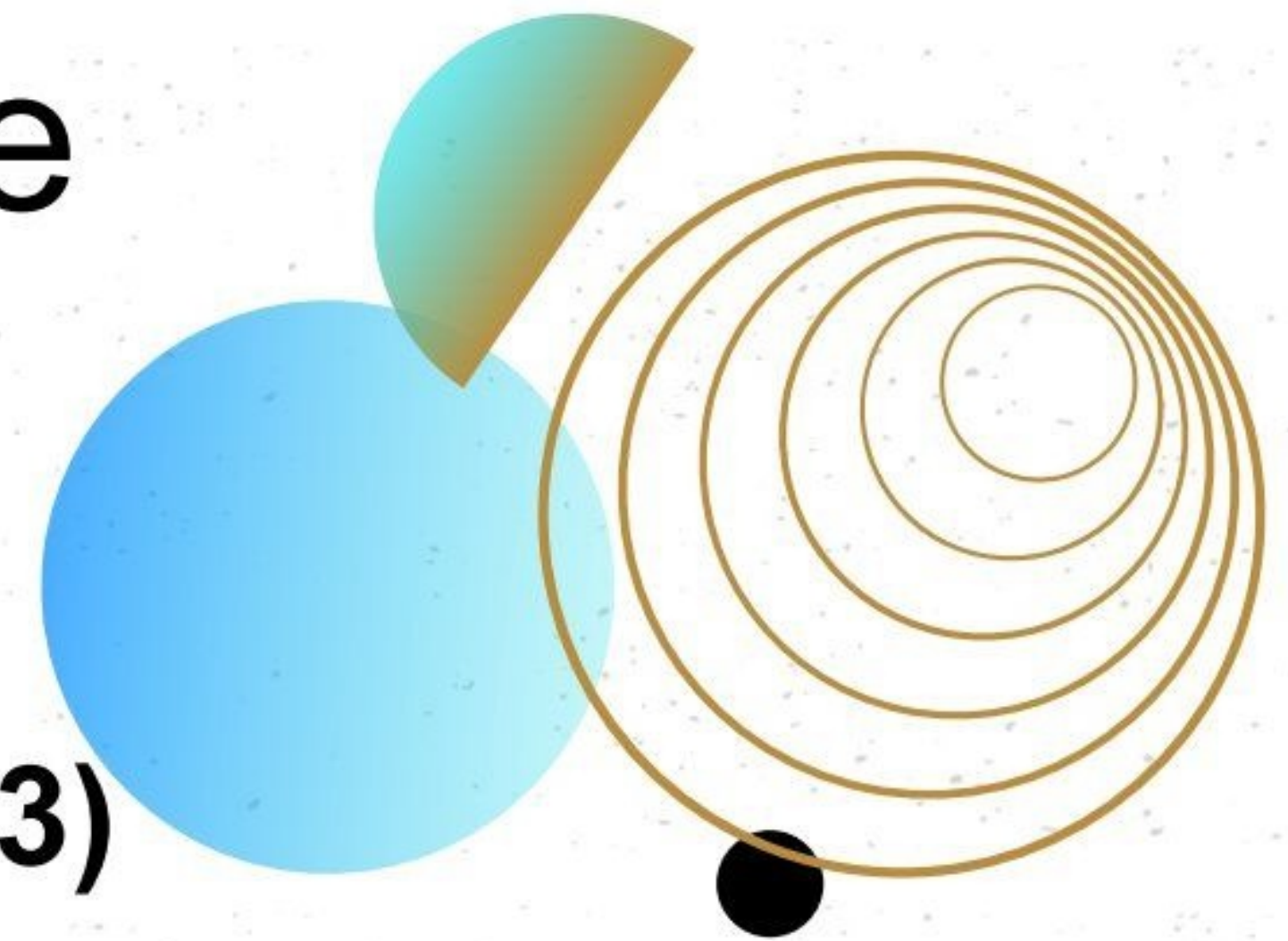




Fibonacci Recursion Tree



< 0 1 >



Countdown Example

Goal:

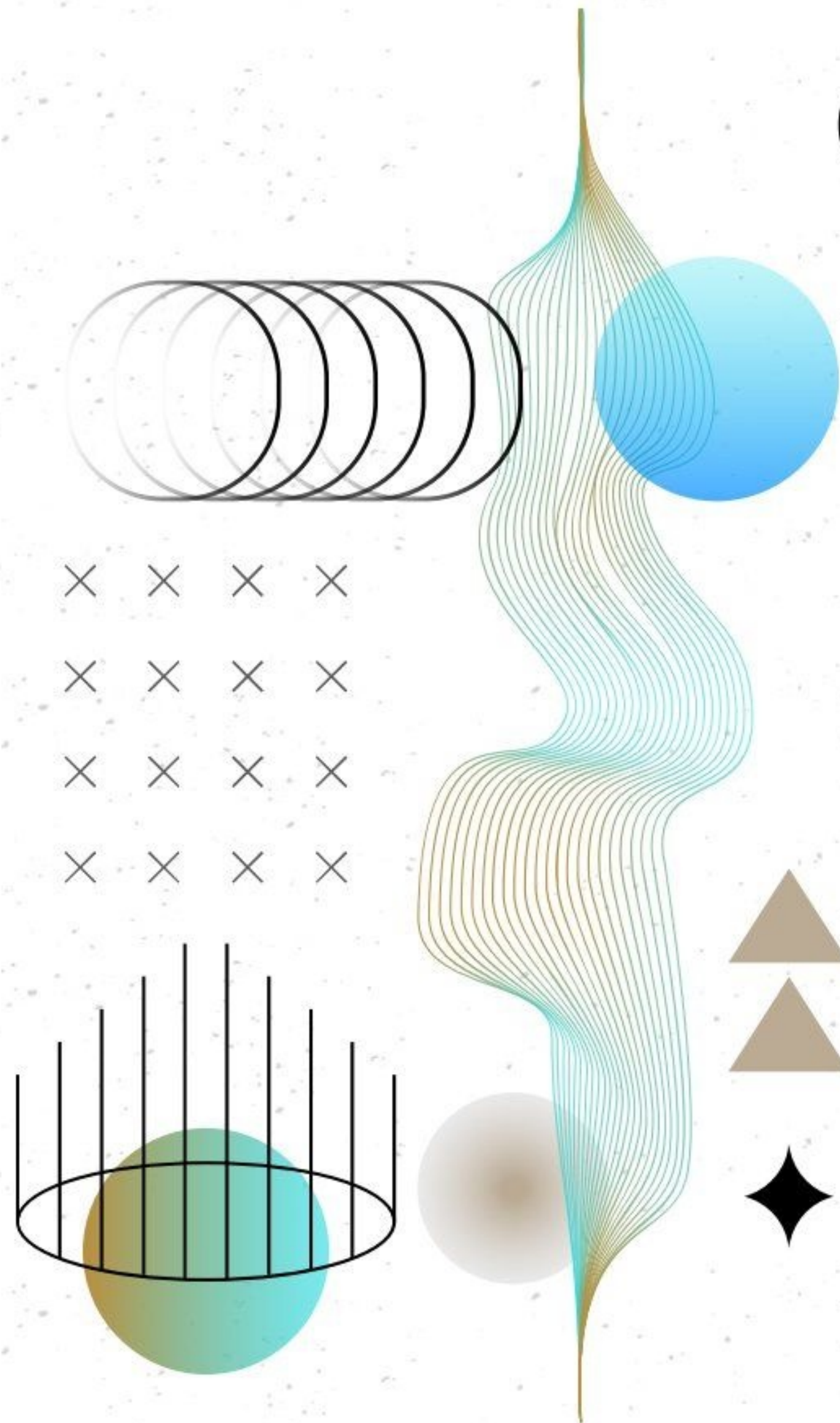
Print numbers from n down to 1

Example:

Countdown(5)

Output:

5,4,3,2,1



Countdown C# Code

```
void Countdown(int n)
```

```
{
```

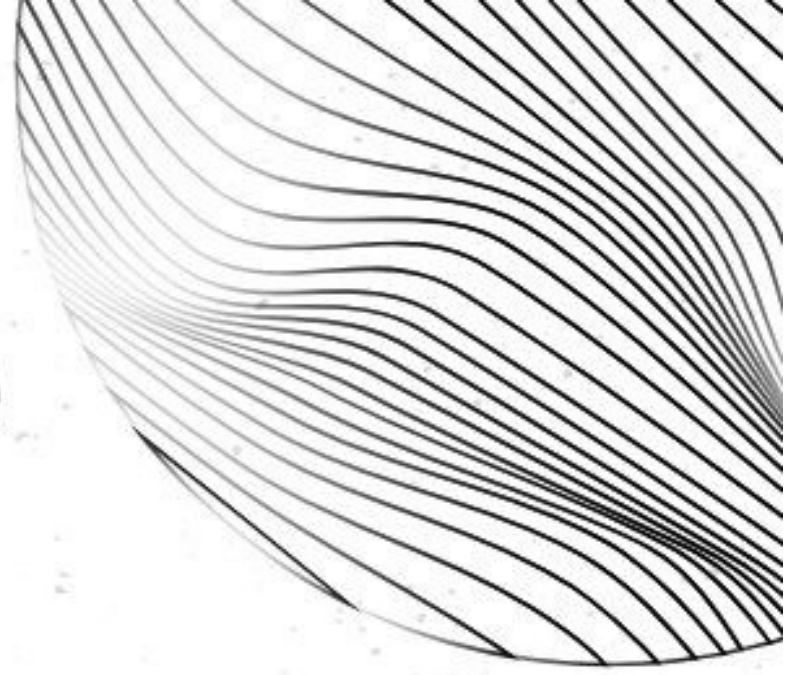
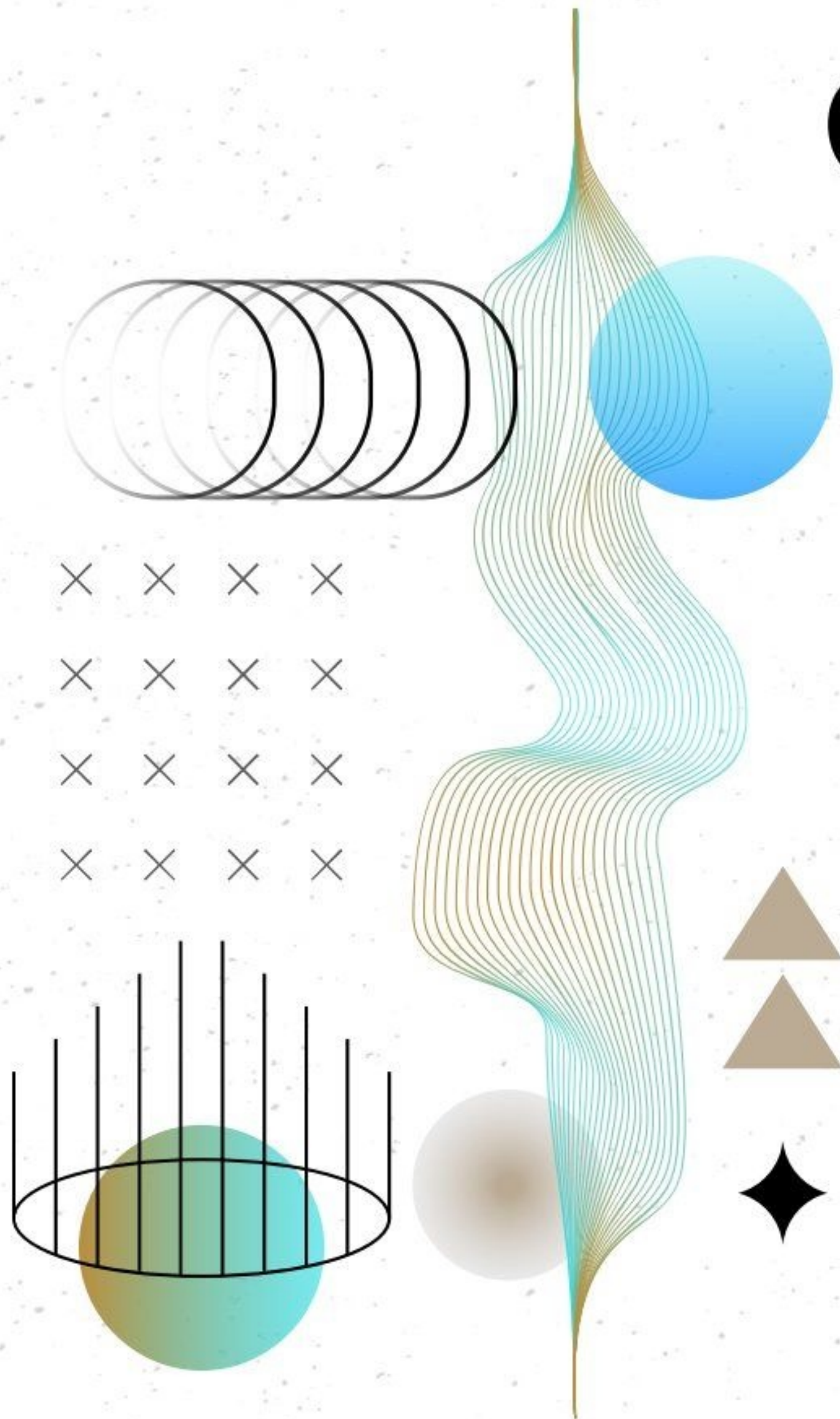
```
    if (n == 0)
```

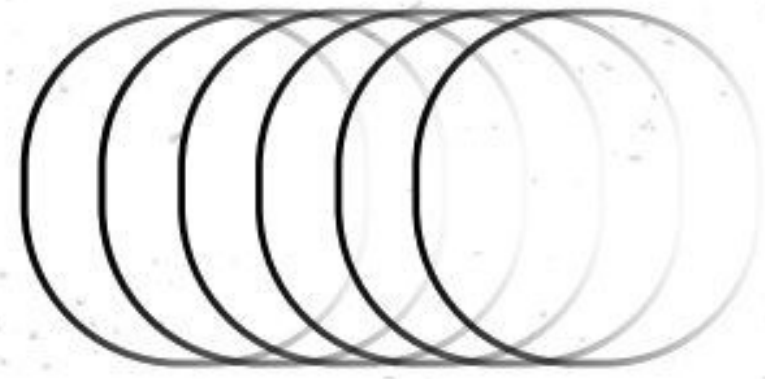
```
        return;
```

```
    Console.WriteLine(n);
```

```
    Countdown(n - 1);
```

```
}
```





Recursion vs Iteration

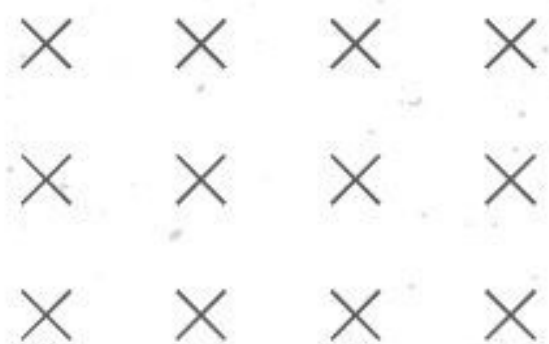
Iteration (Loops)

Uses loops

Usually faster

Lower memory use

Easier to optimize



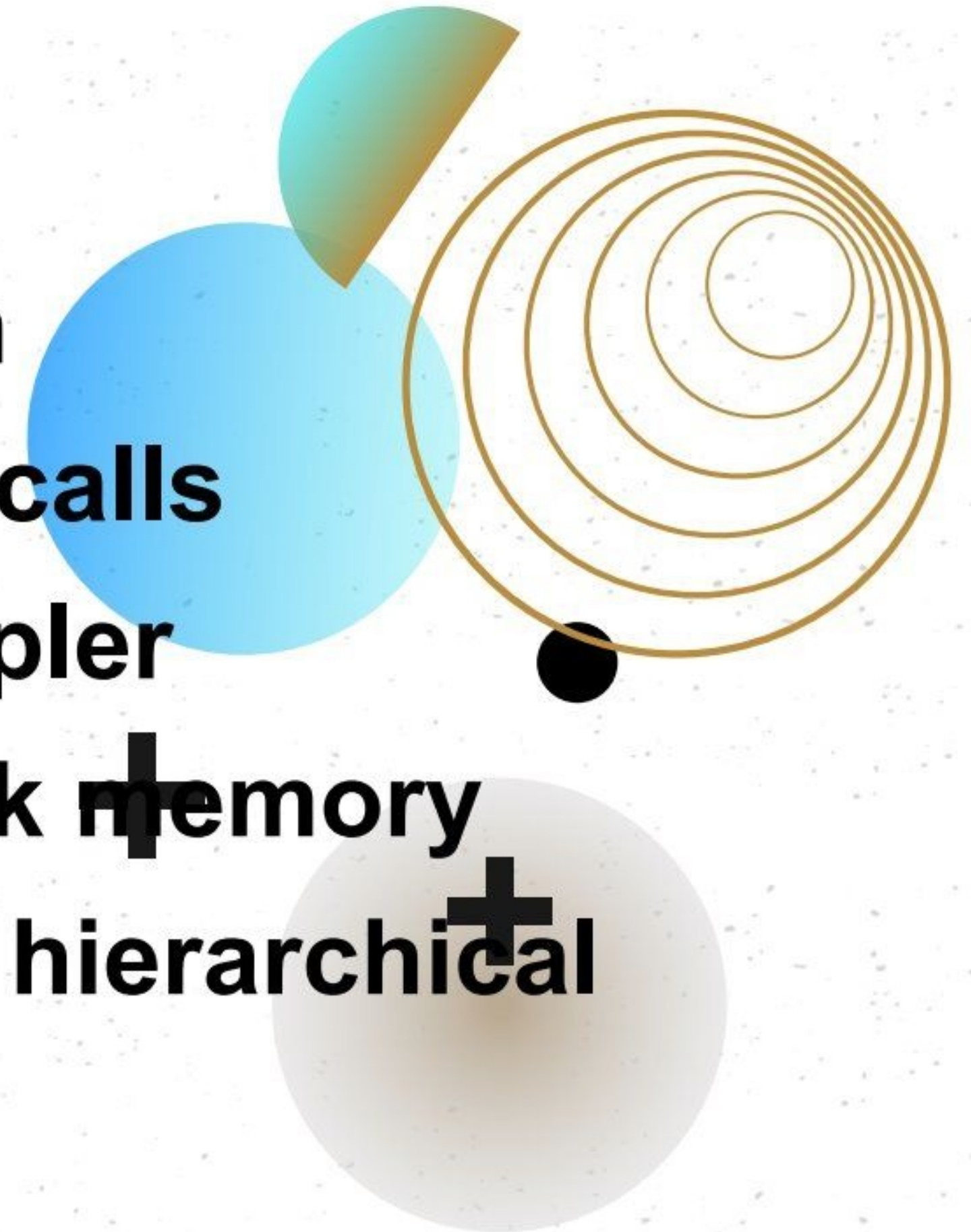
Recursion

Uses self-calls

Often simpler

Uses stack memory

Easier for hierarchical problems



Advantages & Limitations

Advantages of Recursion:

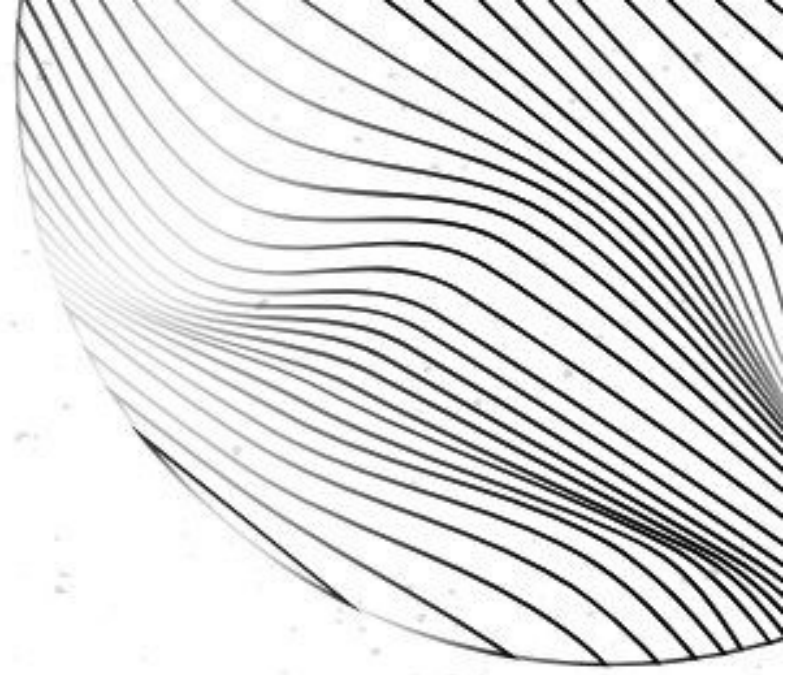
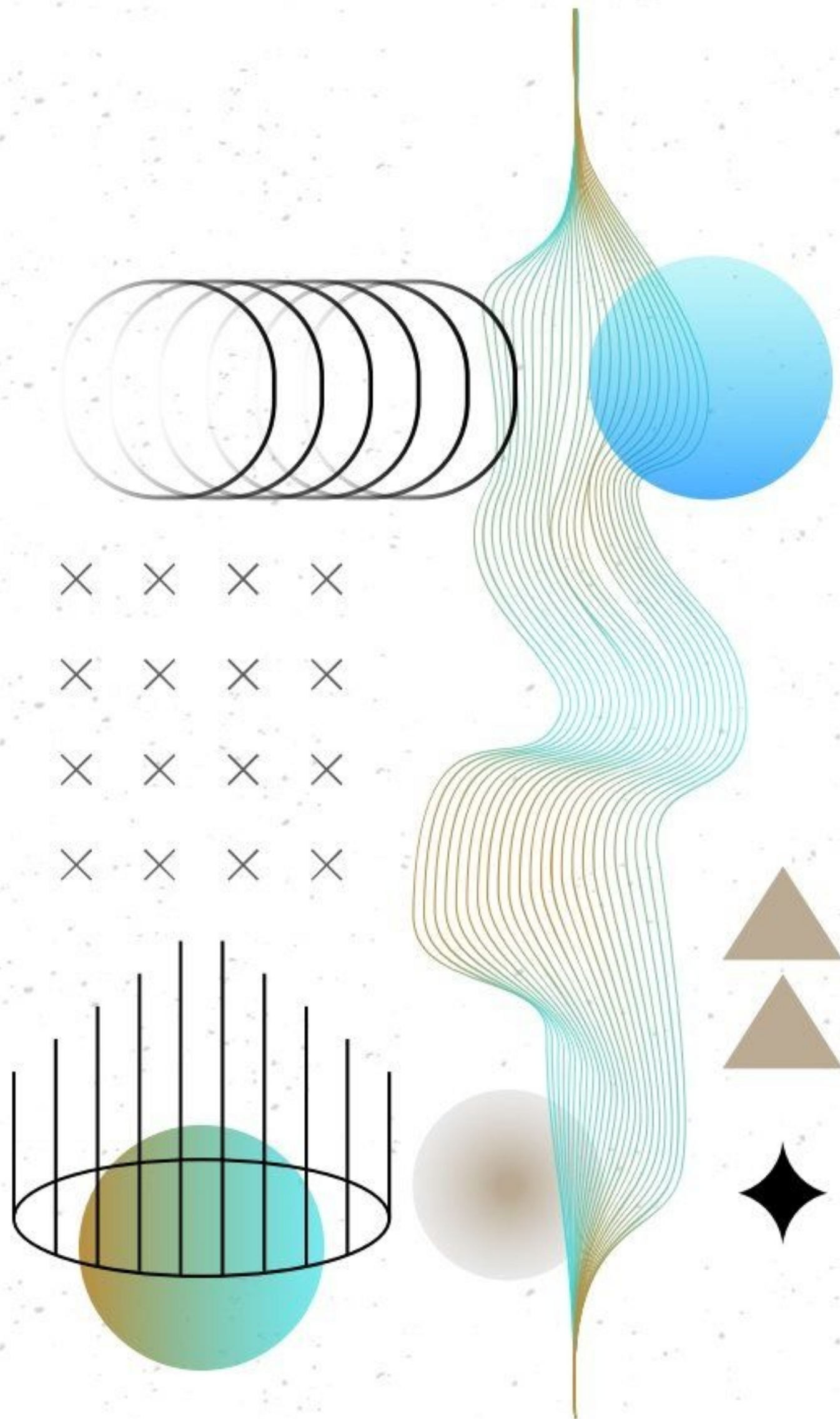
- Cleaner code for some problems
- Natural for trees and graphs
- Simplifies divide & conquer problems

Limitations:

- Uses more memory
- Can cause stack overflow
- May repeat calculations

Real Applications of Recursion

-  Tree traversal
 -  Depth First Search (DFS)
 -  File system navigation
 -  Divide & Conquer algorithms
 -  Game decision trees



Common Recursion Mistakes

- Missing base case
- Incorrect stopping condition
- Infinite recursion
- Large memory usage
- Repeated calculations



Can lead to Stack Overflow

